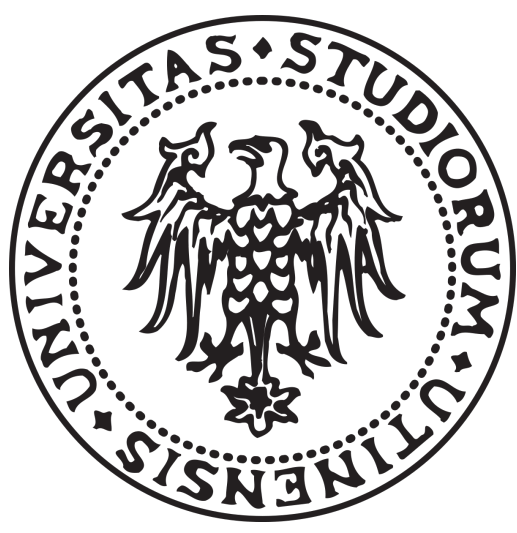




INGEGNERIA DEL SOFTWARE – 2025–26

TESTING BLACK BOX E WHITE BOX

LEZIONE 24
22/12/2025
VINCENZO RICCIO

RIFERIMENTI

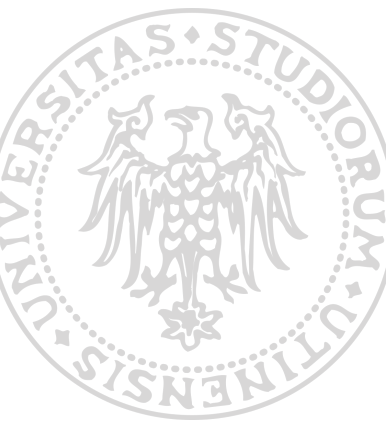
- Sommerville - Capitolo 8

Vincenzo Riccio
Ingegneria del Software 2025/2026
Università degli Studi di Udine





- Vi sono due approcci principali al testing:
 - ◎ **Black box:** Conoscendo le specifiche funzionali, si progettano casi di test per dimostrare che ciascuna funzionalità è completamente operativa, ignorando i dettagli interni del software
 - ◎ **White box:** Conoscendo il funzionamento interno dei vari componenti del software, si progettano casi di test per verificare che meccanismi interni funzionino bene, cioè che le operazioni interne siano svolte correttamente

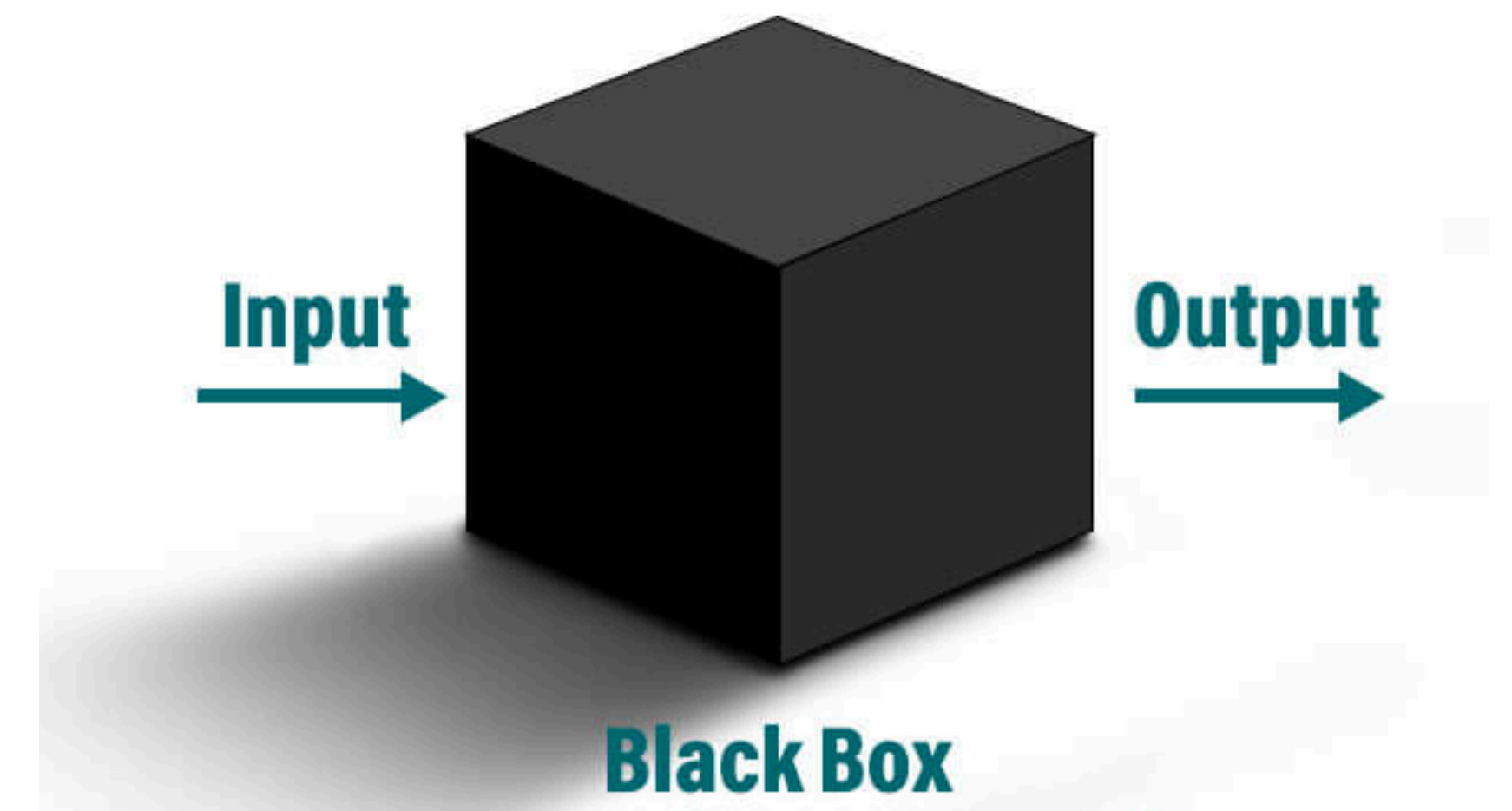


TESTING BLACK BOX

TESTING BLACK BOX



- Il sistema appare come una "scatola nera" poiché la struttura interna non è visibile al tester
- Approccio particolarmente utile quando non si ha accesso al codice sorgente
- Esercita il sistema immettendo input ed osservando i valori degli output
- Sfrutta la conoscenza di:
 - ⦿ Interfaccia del sistema
 - ⦿ Documentazione, ad es. descrizione degli scenari
- Non tiene conto di:
 - ⦿ Codice sorgente
 - ⦿ Stato interno dell'applicazione

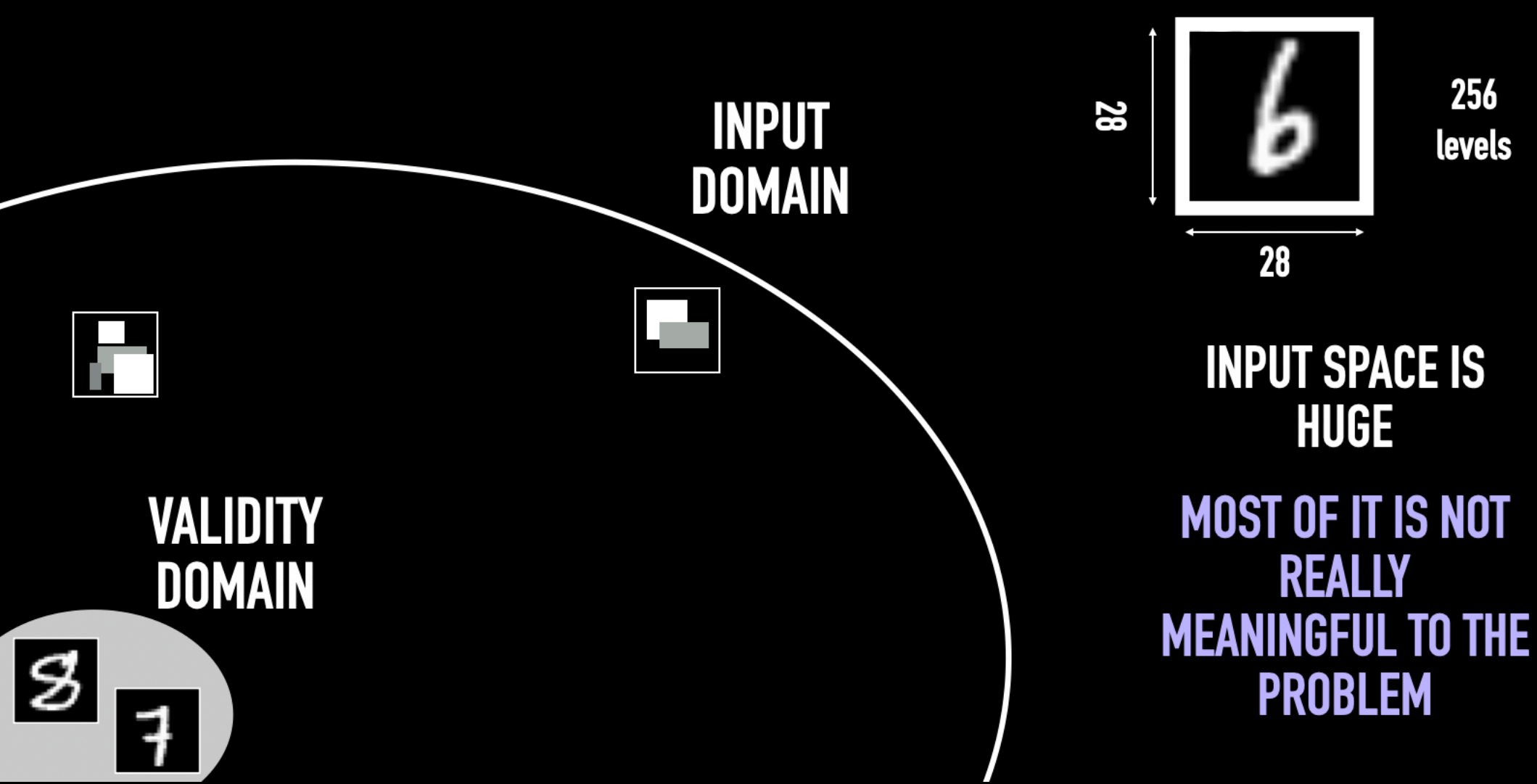


CRITERI DI COPERTURA PER IL TESTING BLACK BOX



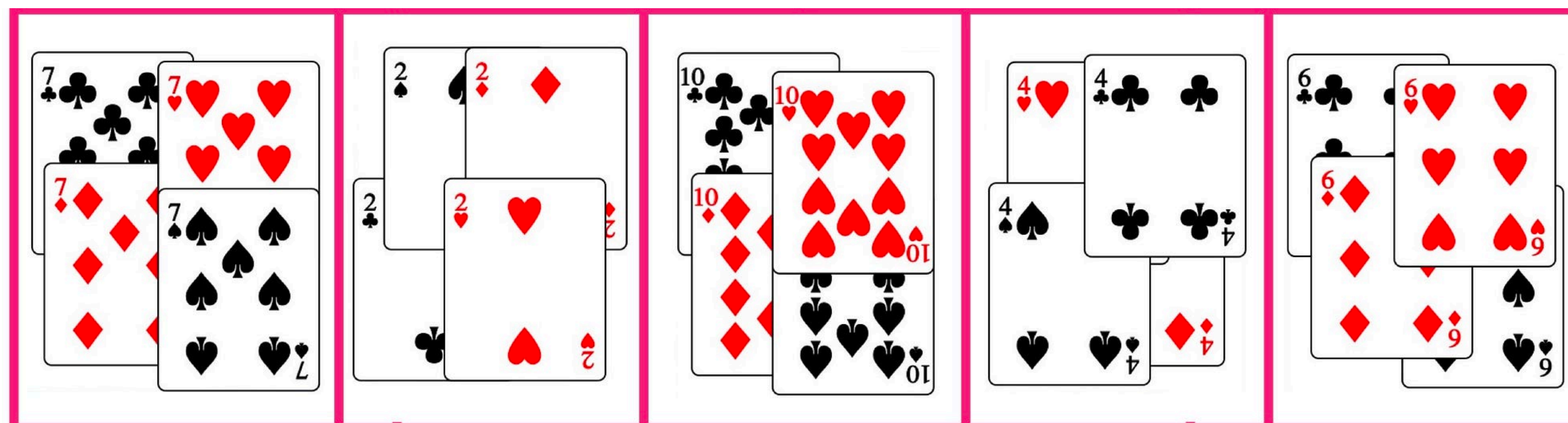
- Obiettivi principali per il testing black box possono essere
 - ◎ Copertura degli scenari di esecuzione definiti nei casi d'uso
 - ◎ Copertura delle funzionalità definite nella specifica dei requisiti
- Il problema è che lo spazio dei possibili input è estremamente ampio e quindi bisogna sceglierne un sottoinsieme significativo

SEMANTIC MANIFOLD PROBLEM



Il problema della selezione degli input di test è ancora più critico quando determinare la validità di tali input è complesso o richiede il giudizio di esperti di dominio

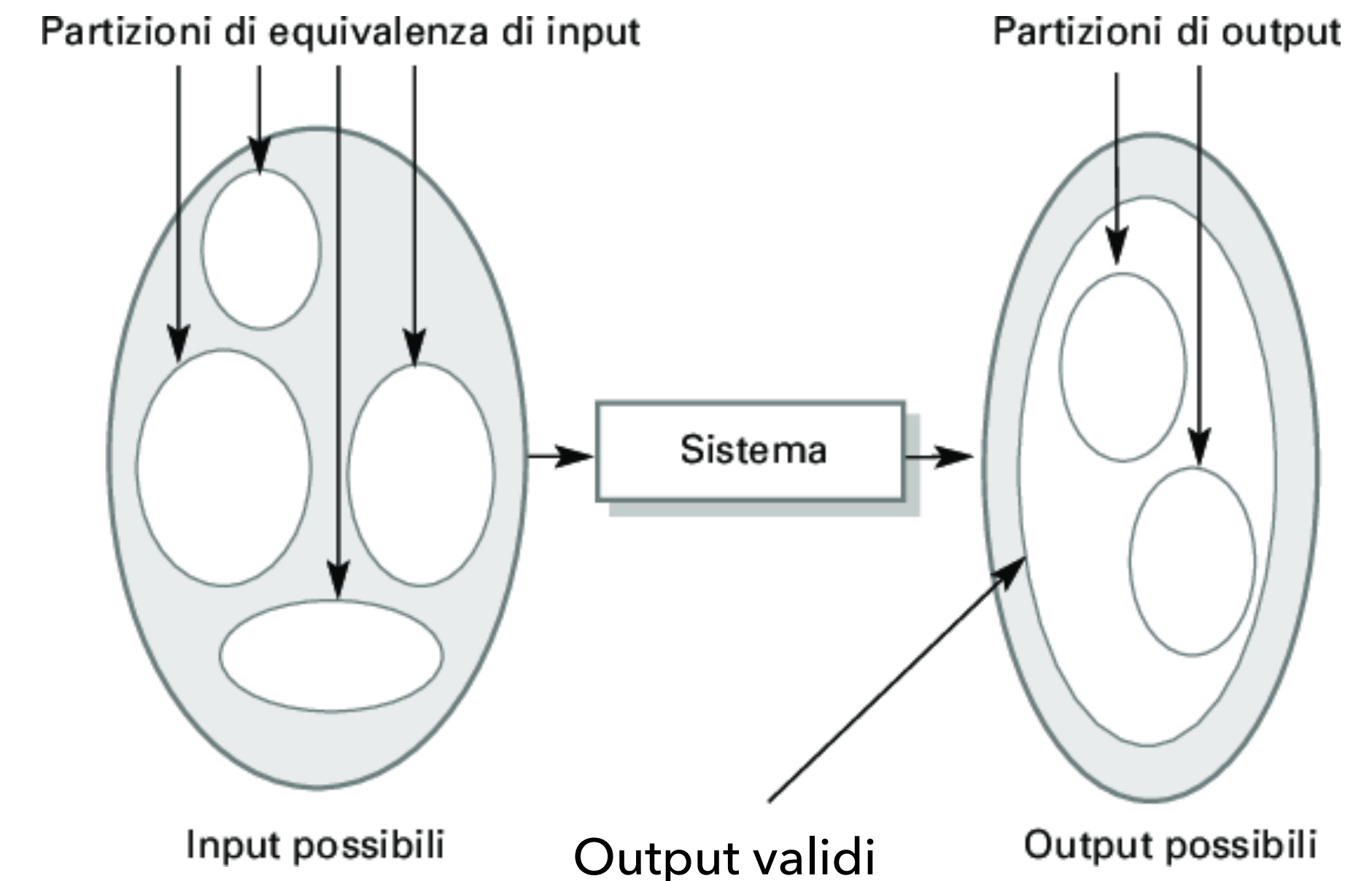
- **Partizionamento in classi di equivalenza:** in base ai requisiti del software testato, si suddivide lo spazio degli input in sottoinsiemi detti **classi di equivalenza** i cui elementi (*si ritiene che*) si comporteranno allo stesso modo durante l'esecuzione
 - Per definizione, la suddivisione in classi di equivalenza è una partizione, quindi ogni intersezione tra classi è vuota
- Criterio di **copertura delle classi di equivalenza:** I casi di test eseguiranno almeno un test per ogni classe di equivalenza



SUDDIVISIONE IN CLASSI DI EQUIVALENZA



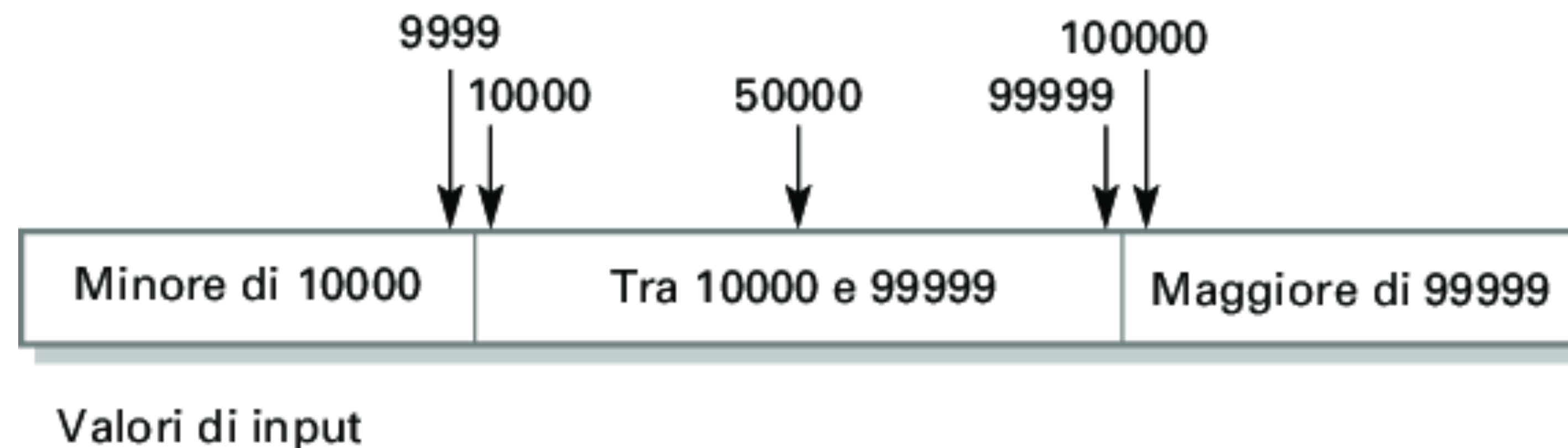
- ▶ Anche lo spazio degli output può essere diviso in partizioni di equivalenza, ossia partizioni in cui il sistema avrà comportamento simile
- ▶ Si può considerare la caratteristica di validità dell'input a esercitare una particolare funzionalità del software definita nei requisiti
- ▶ Si definiranno una o più classi di **Input Validi** e una o più classi di **Input Non Validi** (che dovrebbero essere gestiti da eccezioni)



SUDDIVISIONE IN CLASSI DI EQUIVALENZA: ESEMPIO



- **Esempio:** condizione sulle variabili di ingresso
- La variabile di input è valida se appartiene ad un intervallo di valori, allora avremo input interni all'intervallo (validi) e input esterni all'intervallo (non validi)
- Se dettagliamo ulteriormente la classe degli input non validi abbiamo:
 - Una classe valida per valori interni all'intervallo,
 - Una classe non valida per valori inferiori al minimo
 - Una classe non valida per valori superiori al massimo



CLASSI DI EQUIVALENZA – INTERVALLO DI VALORI



- Esempio: Se la variabile di input rappresenta il voto ad un esame universitario ritenuto valido solo se nell'intervallo tra 18 e 30, possiamo considerare tre classi di equivalenza (CE):
 - CE1 = {valori ≥ 18 e ≤ 30 } (classe di valori validi);
 - CE2 = {tutti i valori < 18 } (non validi)
 - CE3 = {tutti i valori > 30 } (non validi)
- Una possibile test suite TS che copra tutte le classi di equivalenza ha 3 casi di test con i seguenti input per la variabile voto: $\text{Input}_{\text{TS}} = \{1, 20, 40\}$ che "**coprono**" tutte le classi di equivalenza CE1, CE2, CE3

CLASSI DI EQUIVALENZA – INSIEME DISCRETO



- Se la variabile di input valida è un elemento di un insieme discreto, avremo le seguenti classi:
 - ⊙ Una classe valida per ogni elemento dell'insieme $CE1, CE2, \dots, CEn$
 - ⊙ Una classe non valida per elementi non appartenenti all'insieme
- Es. variabile colore di un semaforo, copertura delle classi di equivalenza
- $Input_{TC1} = \text{Rosso} (CE1: \text{Valida})$
- $Input_{TC2} = \text{Verde} (CE2: \text{Valida})$
- $Input_{TC3} = \text{Giallo} (CE3: \text{Valida})$
- $Input_{TC4} = \text{Bianco} (CE4: \text{Non Valida})$

CLASSI DI EQUIVALENZA – TIPO BOOLEANO



- Se la variabile di input è un elemento di tipo booleano, avremo le seguenti classi:
 - Una classe CE1 valida costituita dall'elemento con valore `True`
 - Una classe CE2 non valida costituita da un elemento con valore `False`

CLASSI DI EQUIVALENZA – TIPO NON VALIDO



- In ognuno dei casi precedenti, bisogna aggiungere anche un'ulteriore classe non valida, composta da elementi che non appartengono al tipo della variabile di input
- Tale classe verifica se il programmatore ha effettuato gli opportuni controlli sul tipo di dato
- Es.: Se la variabile è di tipo intero, allora si consideri la classe (non valida) di valori di tipo carattere

BLACK BOX TESTING – ESERCIZIO



- Un metodo riceve in input una data composta da:
 - ⊙ Giorno (numerico compreso tra 1 e 31),
 - ⊙ Mese (stringa che può valere {'gennaio', ..., 'dicembre'})
 - ⊙ Anno (numerico compreso tra 1900 e 2000)
- Restituisce in output il giorno della settimana corrispondente, variabile numerica (1 = lunedì, 2 = martedì, ..., 7 = domenica)
- Selezionare i casi di test mediante partizione delle classi di equivalenza
- Consideriamo ciascuna variabile **separatamente: giorno, mese, anno**

BLACK BOX TESTING – ESERCIZIO



- Variabile **Giorno**
- **Condizioni di ingresso**
 - ⊙ Il giorno può essere compreso tra 1 e 31
- **Classi di Equivalenza**
 - ⊙ Valida
 - ⊙ CE1: $1 \leq \text{Giorno} \leq 31$
 - ⊙ Non Valida
 - ⊙ CE2 : $\text{Giorno} < 1$
 - ⊙ CE3 : $\text{Giorno} > 31$
 - ⊙ CE4 : Giorno non intero

BLACK BOX TESTING – ESERCIZIO



- Variabile **Mese**
- **Condizioni di ingresso**
 - ⊙ Il mese deve essere compreso nell'insieme $M = \{\text{gennaio}, \dots, \text{dicembre}\}$, per semplicità non consideriamo ogni membro dell'insieme
- **Classi di Equivalenza**
 - ⊙ Valida
 - ⊙ CE5: $\text{Mese} \in M$
 - ⊙ Non Valida
 - ⊙ CE6: Mese stringa di caratteri $\notin M$
 - ⊙ CE7: Tipo non corretto, ad es. intero

BLACK BOX TESTING – ESERCIZIO



- Variabile **Anno**
- **Condizioni di ingresso**
 - ⊙ L'anno è un intero compreso tra 1900 e 2000
- **Classi di Equivalenza**
 - ⊙ Valida
 - ⊙ CE8: $1900 \leq \text{Anno} \leq 2000$
 - ⊙ Non Valida
 - ⊙ CE9 : $\text{Anno} < 1900$
 - ⊙ CE10 : $\text{Anno} > 2000$
 - ⊙ CE11 : Anno non intero

BLACK BOX TESTING – ESERCIZIO



TC	TC1	...	TCn
ID			
Breve descrizione			
Precondizione			
Variabile Input 1			
Variabile Input 2			
Variabile Input 3			
Output Atteso			
Postcondizione Attesa			
CE			

- Dobbiamo preparare una tabella che contenga tutte le informazioni del caso di test
- Per semplicità focalizziamoci per ora solo su input e CE coperte

BLACK BOX TESTING – ESERCIZIO



TC	TC1	TC2	TC3	TC4	TC5	TC6	TC7	TC8	TC9
Giorno	1	1	1	1	1	1	0	35	Primo
Mese	Gennaio	Gennaio	Gennaio	Gennaio	Brumaio	1234	Gennaio	Gennaio	Gennaio
Anno	1980	1492	2018	Duemila	1980	1980	1980	1980	1980
CE	1, 5, 8	1, 5, 9	1, 5, 10	1, 5, 11	1, 6, 8	1, 7, 8	2, 5, 8	3, 5, 8	4, 5, 8

- TS che copre tutte le **combinazioni di classi di equivalenza**
- Ogni TC riesce a coprire una classe di equivalenza non coperta da alcuno dei precedenti
- La TS copre tutte le classi di equivalenza ma non tutte le possibili combinazioni di input (rimane possibile definire il 30 febbraio)
- Progettare test in grado di coprire tutte le combinazioni ammissibili avrebbe aumentato l'efficacia (malfunzionamenti trovati) ma ridotto l'efficienza

BLACK BOX TESTING – ESERCIZIO



TC	TC1	TC2	TC3	TC4
Giorno	1	0	35	Primo
Mese	Gennaio	Brumaio	1234	Gennaio
Anno	1980	1492	2018	Duemila
CE	1, 5, 8	2, 6, 9	3, 7, 10	4, 5, 11

- **TS minima che copre tutte le CE**
- TS più **efficiente** poiché copre tutte le classi con meno test case
- Più difficile rilevare difetti a causa del **masking**: in TC2 il sistema potrebbe sollevare un'eccezione perché $\text{giorno} < 1$ senza valutare mese e anno e quindi non sapremmo come il programma si comporterebbe in quei casi

CLASSI DI EQUIVALENZA – BOUNDARY VALUES



- L'esperienza insegna che i programmatori fanno spesso errori per valori di input che sono vicini ai confini (boundaries) delle classi di equivalenza
- Bisogna testare il software anche con valori di input che coincidono (o sono vicini) ai valori limite delle classi di equivalenza.
- Non ci si limiterà a scegliere solo valori interni alla classe di equivalenza
- Tale criterio si applica efficacemente a sottoinsiemi di insiemi continui (interi, reali) e, in particolare, ad intervalli di valori

CLASSI DI EQUIVALENZA – BOUNDARY VALUES



- Per ciascun estremo dell'intervallo, 1 CE per l'estremo e 2 CE per i valori sulla frontiera
- Per l'input anno (le specifiche del problema imponevano anno compreso tra 1900 e 2000)
 - 1899: valore leggermente inferiore dell'estremo inferiore dell'intervallo
 - 1900: estremo inferiore
 - 1901: valore leggermente superiore all'estremo inferiore
 - 1950: valore nominale
 - 1999: valore leggermente inferiore all'estremo superiore
 - 2000: estremo superiore
 - 2001: valore leggermente maggiore dell'estremo superiore



- A volte non é possibile determinare a priori gli input per coprire le classi di equivalenza ma bisogna conoscere le precondizioni
- Ad esempio per una variabile password
 - **Classi valide**
 - CE1: password corrispondente a un utente con diritto di accesso
 - **Classi non valide**
 - CE2: password corrispondente a un utente che non ha diritto di accesso
 - CE3: password vuota
- In tal caso bisogna specificare la **precondizione** in modo da tenere conto di quale password corrisponde a un utente con diritto di accesso.



ALTRI ESERCIZI SU TESTING BLACK BOX



- Si vuole testare un'applicazione per compilare liste di cose da fare (to-do lists). In particolare, si vuole testare la funzionalità aggiungi nuovo compito. Si adotti una strategia di testing a livello di sistema con approccio black-box. L'input deve essere una stringa alfanumerica di lunghezza compresa tra 2 e 10 caratteri. Se l'input è valido, il sistema aggiunge il to-do nella lista. Altrimenti, mostra il messaggio di errore.
- **Progettare un'opportuna test suite mediante partizione delle classi di equivalenza che tenga conto anche dei valori boundary.**



Abbiamo una sola variabile **compito** di tipo **stringa alfanumerica**

- CE1 = {stringhe con # caratteri == 1} (non valida)
- CE2 = {stringhe con # caratteri == 2} (valida)
- CE3 = {stringhe con # caratteri == 3} (valida)
- CE4 = {stringhe con # caratteri ≥ 4 e ≤ 8 } (valida)
- CE5 = {stringhe con # caratteri == 9} (valida)
- CE6 = {stringhe con # caratteri == 10} (valida)
- CE7 = {stringhe con # caratteri == 11} (non valida)
- CE8 = tipo non valido (accetta tutti i caratteri, ma non gestisce il campo vuoto)



ID	Descrizione	Precondizioni	Valori di Input	Output e postcondizioni attesi	CE coperta
1	1 lettera (Boundary)	To-do prompt presente	"A"	Messaggio di errore ed elemento non inserito	1
2	2 lettere (estremo inf)	To-do prompt presente	"Di"	Elemento inserito	2
3	3 lettere (Boundary)	To-do prompt presente	"Tre"	Elemento inserito	3
4	4 lettere	To-do prompt presente	"Four"	Elemento inserito	4
5	9 lettere (Boundary)	To-do prompt presente	"Esplorare"	Elemento inserito	5
6	10 lettere (estremo sup)	To-do prompt presente	"Abbronzare"	Elemento inserito	6
7	11 lettere (Boundary)	To-do prompt presente	"Trasformare"	Messaggio di errore ed elemento non inserito	7
8	Tipo non valido	To-do prompt presente	Campo vuoto	Messaggio di errore ed elemento non inserito	8

Desideri iscriverti all'esame?

☐ Si ☐ No

☐ Svolta prima esercitazione

☐ Svolta seconda esercitazione

- Si voglia testare questa interfaccia grafica composta da radio button a selezione mutualmente esclusiva (il sistema permette di selezionarne solo una) e due check box che possono essere selezionate indipendentemente l'una dall'altra (il sistema permette di selezionare entrambe). Progettare una test suite con **numero minimo di casi di test che coprano tutte le possibili configurazioni** della GUI

DUE POSSIBILI VALORI: SI
E NO

Desideri iscriverti all'esame?

☐ Si ☐ No

☐ Svolta prima esercitazione

☐ Svolta seconda esercitazione

DUE POSSIBILI VALORI:
SELEZIONATO E NON
SELEZIONATO

DUE POSSIBILI VALORI:
SELEZIONATO E NON
SELEZIONATO

- In questa GUI tutte le configurazioni sono valide, non ci sono input non validi
- Input TC1 = {W1 = Si, W2 = selezionato, W3 = selezionato},
Postcondizione attesa: studente iscritto
- Input TC2 = {W1 = No, W2 = non selezionato, W3 = non selezionato},
Postcondizione attesa: studente non iscritto

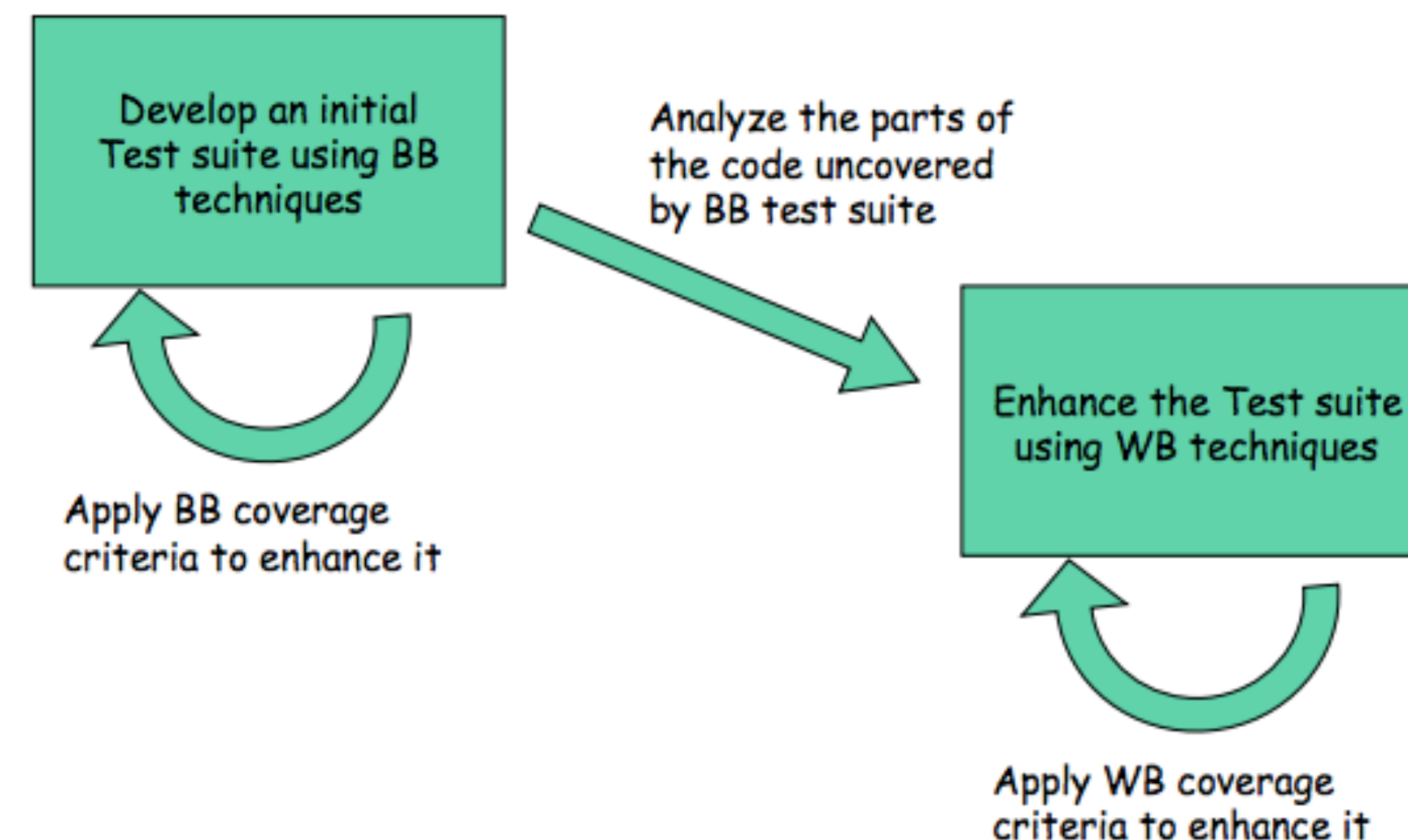


TESTING WHITE BOX

TESTING STRUTTURALE (WHITE BOX)



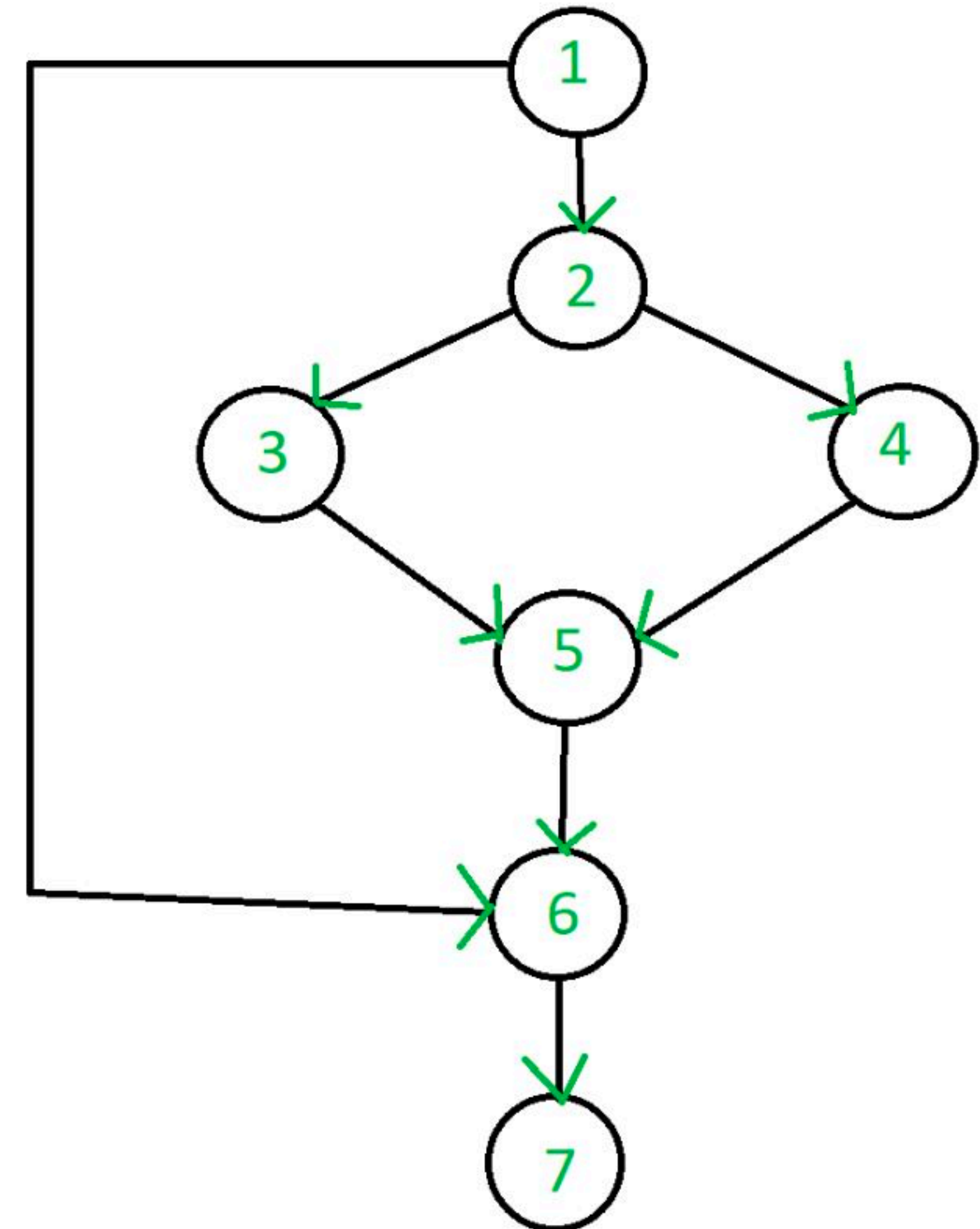
- Utilizza la struttura interna del software per ricavare i dati di test
 - Differenza dal testing black box
- I test white box possono fornire maggiori informazioni al debugger sulla posizione dell'errore
- Complementa il testing black box: le "parti" di programma non testate dal testing black box potrebbero contenere errori



RAPPRESENTAZIONE GRAFICA DEL CODICE TRAMITE CFG



- Il **Control Flow Graph (CFG)** permette di rappresentare la struttura di un programma P
- Grafo orientato che rappresenta il trasferimento del flusso di controllo tra blocchi di istruzioni del programma



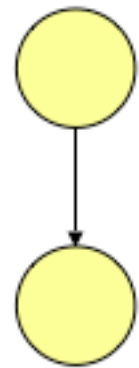


- Un **nodo** rappresenta un blocco di una o più istruzioni, tutte eseguite se il flusso di controllo raggiunge la prima di esse
- Un **arco** rappresenta il trasferimento del flusso di controllo tra la coppia di nodi che esso collega
- Un CFG ha sempre due nodi speciali: nodo di ingresso e di uscita
- I nodi possono appartenere a uno dei due insiemi disgiunti:
 - **Nodi istruzione:** hanno un solo arco uscente
 - **Nodi predicato:** corrispondono a condizioni, hanno due archi uscenti etichettati con Vero o Falso o più archi uscenti nel caso di switch-case (che potrebbe essere anche rappresentato come cascata di if)

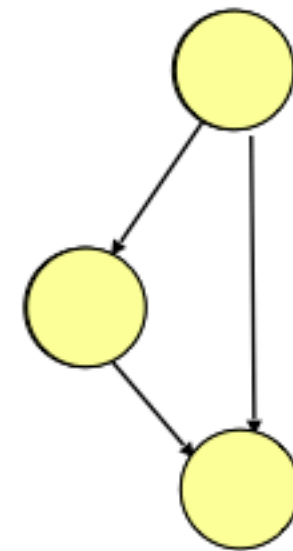
CFG: ISTRUZIONI PER IL CONTROLLO DI FLUSSO



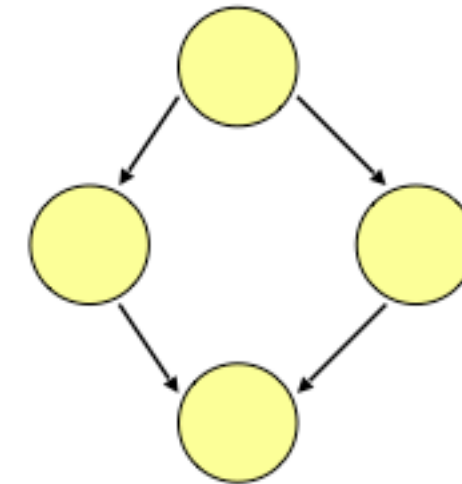
Sequenza



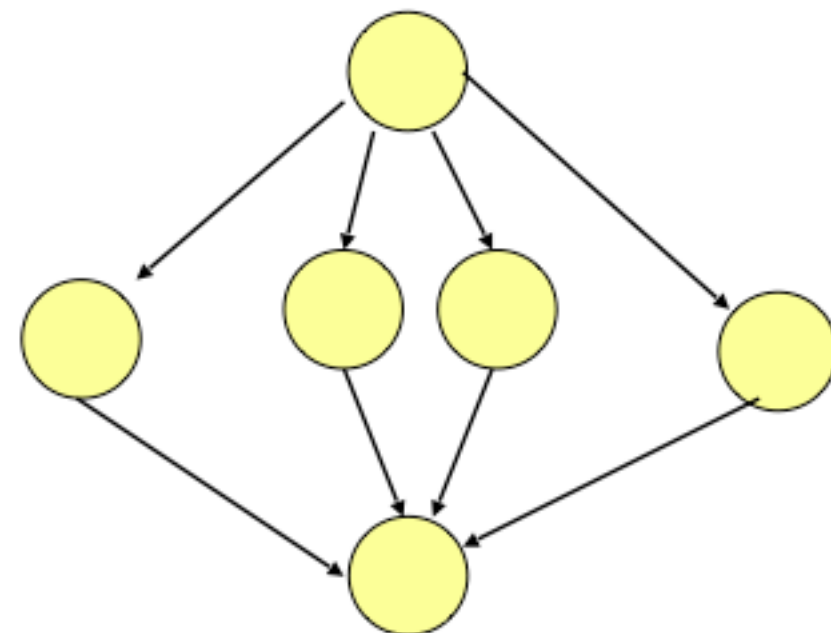
If..then



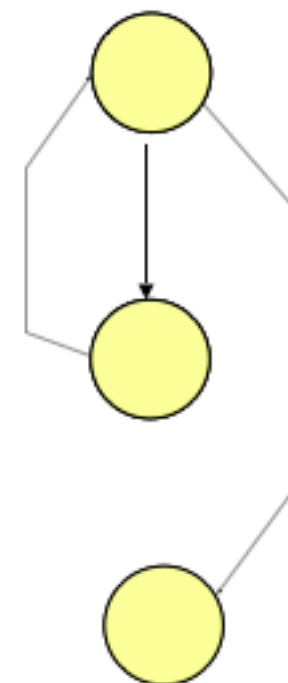
If..then..else



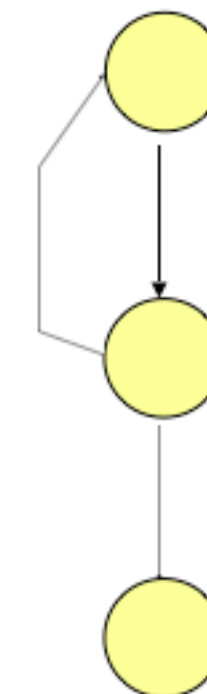
Case



While..do



Repeat..Until



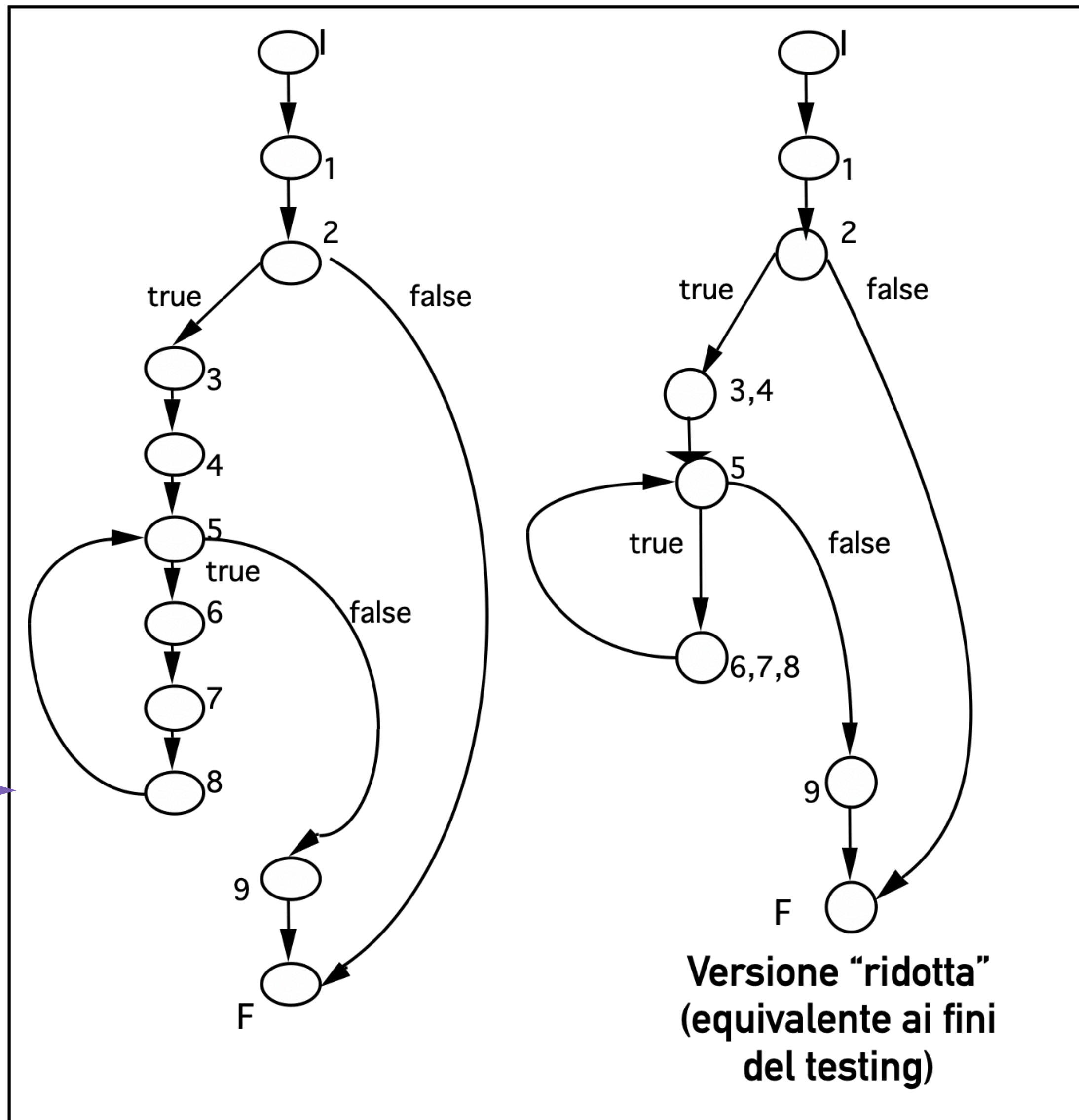
IL CICLO FOR PUÒ ESSERE
RAPPRESENTATO COME UN
WHILE-DO AGGIUNGENDO
UN NODO PER
L'INIZIALIZZAZIONE DELLA
VARIABILE E UN ALTRO PER
L'INCREMENTO

DA CODICE A CFG



```
int Quadrato (int x){  
1.  read(x);  
2.  if (x > 0){  
3.      n = 1;  
4.      y = 1;  
5.      while (x > 1) {  
6.          n = n + 2;  
7.          y = y + n;  
8.          x = x - 1;  
9.      }  
    write(y);  
}
```

- ▶ LE ISTRUZIONI ESEGUITE SEMPRE IN SEQUENZA POSSONO ESSERE RAGGRUPPATE IN UN UNICO BLOCCO
- ▶ **ATTENZIONE AI RAGGRUPPAMENTI:** 5 È UN PREDICATO E NON PUÒ ESSERE RAGGRUPPATO CON LE ISTRUZIONI 3 E 4



CRITERI DI COPERTURA DEL FLUSSO DI CONTROLLO

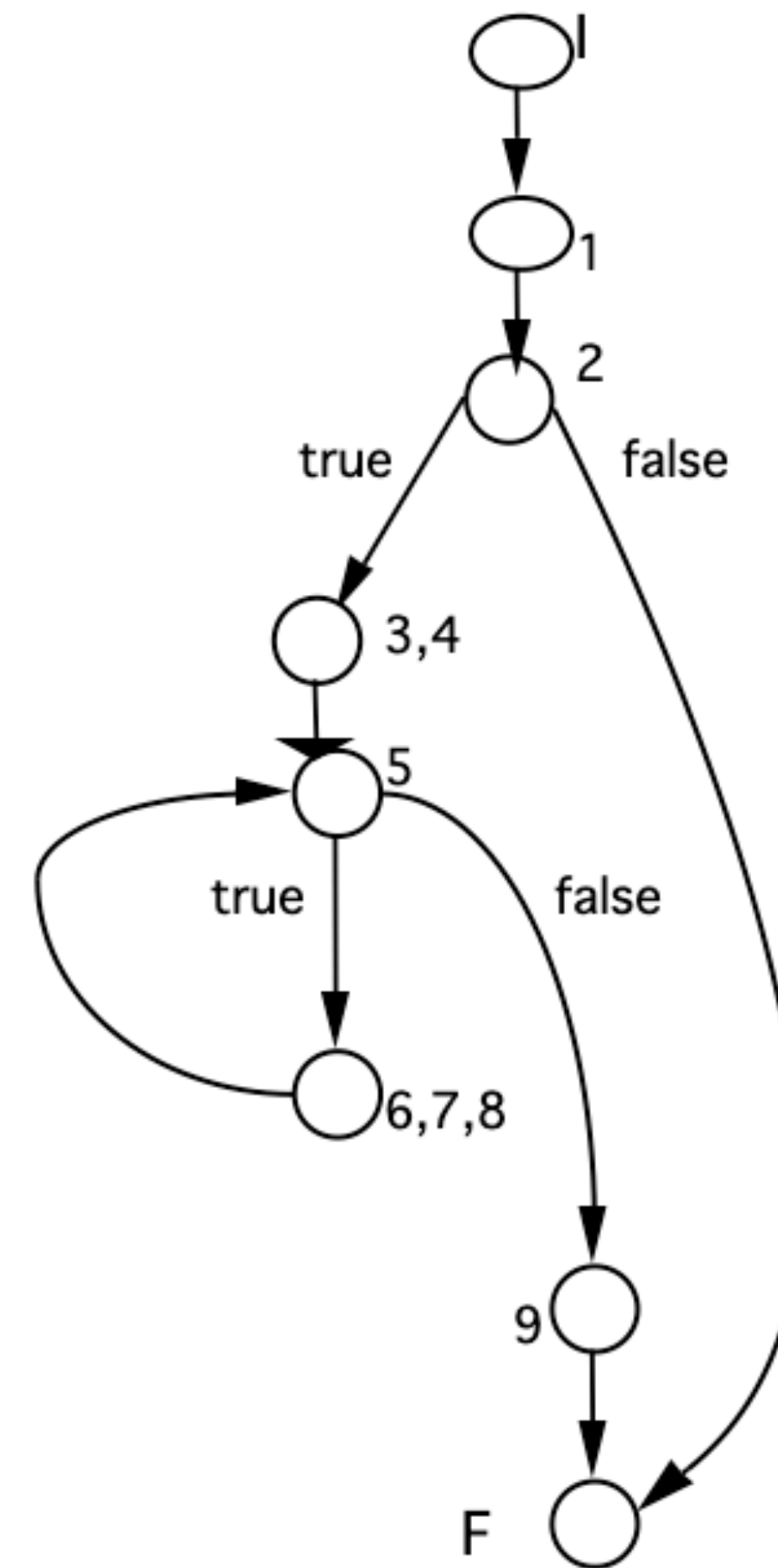


- Criteri di copertura fondati sull'adozione di metodi di Copertura degli oggetti che compongono la struttura dei programmi
- Bisogna definire un insieme di casi di test aventi dati in input in grado di **coprire** (ossia attraversare) tutti i componenti di interesse del software (es. istruzioni, archi del CFG, predicati, strutture di controllo, etc.) almeno una volta durante l'esecuzione dei test case
- In base ai criteri di copertura del CFG, possiamo avere diversi tipi di coverage testing

COPERTURA DEI COMANDI (STATEMENT COVERAGE)



- Il Criterio di Copertura degli Statement richiede di coprire tutti i nodi del CFG durante il testing
- Eseguire tutte le istruzioni dà una debole garanzia di scoprire i difetti, ma è abbastanza semplice da soddisfare
- Infatti non assicura di coprire sia il ramo true che false delle decisioni



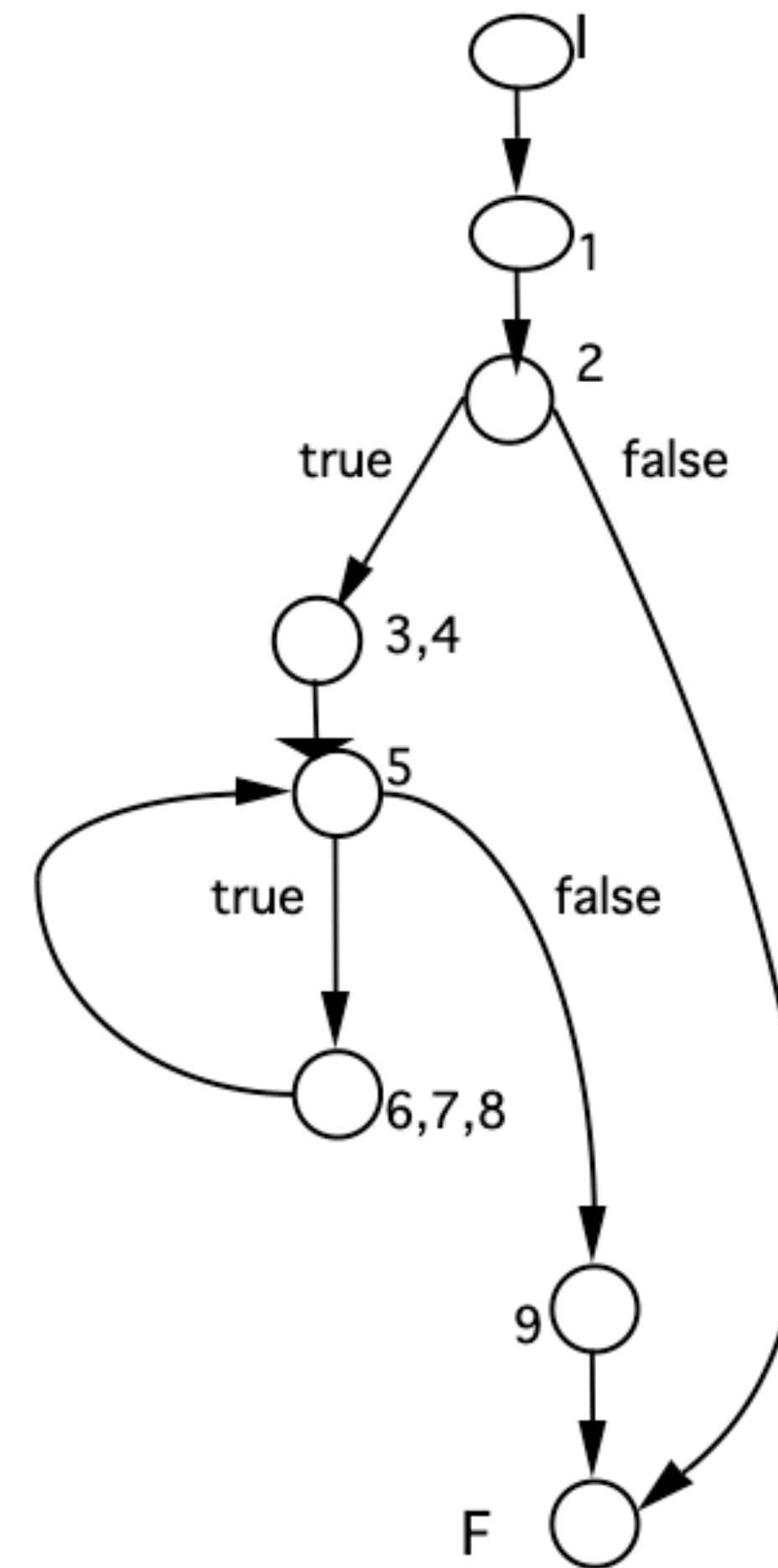
**NELL'ESEMPIO PRECEDENTE
X = 2 GARANTISCE LA
COPERTURA DI TUTTI I NODI
MA NON DELL'ARCO 2-FINE**

**CRITERIO DI ADEGUATEZZA
CORRISPONDENTE:
COMANDI ESEGUITI /
COMANDI ESEGUIBILI**

COPERTURA DELLE DECISIONI (DECISION COVERAGE)



- Richiede che ciascun arco del CFG sia attraversato almeno una volta
- Ogni decisione nel programma (nodi predicato del CFG) deve assumere valore `true` e `false` durante il testing
- Un limite è legato alle decisioni in cui sono valutate più condizioni (legate da operatori logici AND ed OR)



NELL'ESEMPIO
TS = {X = 2; X = -1}
GARANTISCE LA COPERTURA
DELLE DECISIONI

CRITERIO DI ADEGUATEZZA
CORRISPONDENTE:
ARCHI PERCORSI /
ARCHI PERCORRIBILI

LA COPERTURA DELLE
DECISIONI IMPLICA LA
COPERTURA DEI NODI

DECISIONI E CONDIZIONI



- **Decisione:** predicato (espressione) di una istruzione condizionale (if) o di una iterativa (while, for, ...) [ad es., $(x > 0 \mid y > 0)$]
- **Condizione:** espressione atomica booleana che appare in una decisione [ad es., $(x > 0)$ e $(y > 0)$]

DECISION

```
if ( ( x >= 0 ) && ( x <= 200 ) )
```

CONDITION 1 CONDITION 2

COPERTURA DELLE CONDIZIONI (CONDITION COVERAGE)



- Richiede che ciascuna condizione nei nodi decisione di un CFG debba essere valutata sia per valori true che false
- Nell'esempio bisogna assicurare che entrambe le condizioni assumano sia valori true che false

DECISION

```
if ( (x >= 0) && (x <= 200) )
```

CONDITION 1 CONDITION 2

**TS1={5, -5} GARANTISCE
SOLO COPERTURA DECISIONI**

**TS2={-3; 210} GARANTISCE
SOLO COPERTURA
CONDIZIONI**

**LA COPERTURA DELLE
CONDIZIONI NON SEMPRE
IMPLICA LA COPERTURA
DELLE DECISIONI**

COPERTURA DELLE CONDIZIONI E DELLE DECISIONI



- Criterio più forte perché assicura che i casi di test coprono tutte le condizioni ma anche tutte le decisioni

DECISION

```
if ( ( x >= 0 ) && ( x <= 200 ) )
```

CONDITION 1 CONDITION 2

**TS1={5,; -5 } GARANTISCE
SOLO COPERTURA DECISIONI**

**TS2={-3; 210} GARANTISCE
SOLO COPERTURA
CONDIZIONI**

**TS3={-3; 10; 210 }
GARANTISCE COPERTURA
SIA DELLE DECISIONI CHE
DELLE CONDIZIONI**

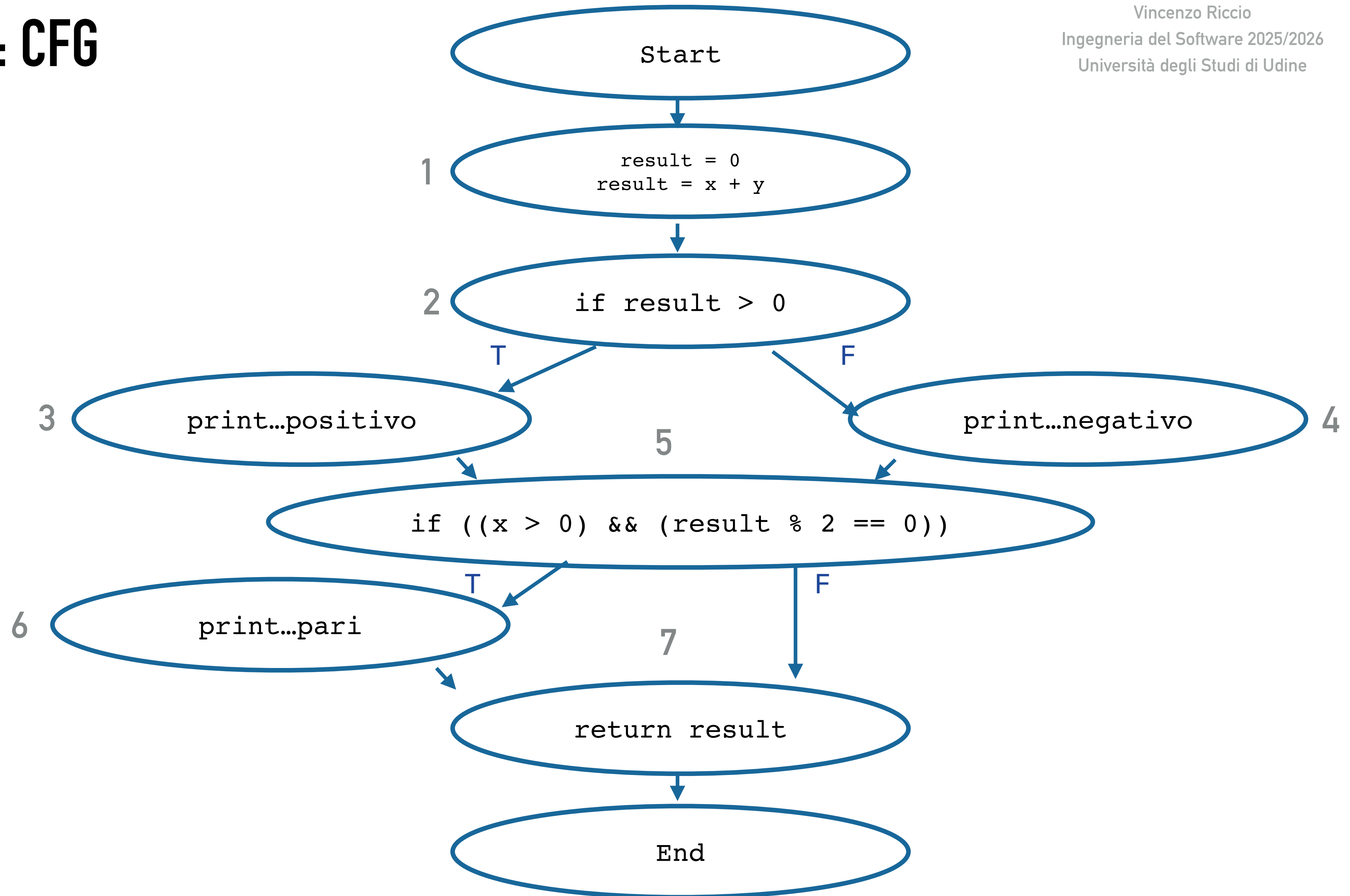


```
def esempio_control_flow(x, y):  
    result = 0  
    result = x + y  
    if result > 0:  
        print("Il risultato è positivo:", result)  
    else:  
        print("Il risultato non è positivo:", result)  
    if ((x > 0) && (result % 2 == 0)):  
        print("Il risultato è un numero pari e il primo addendo positivo.")  
    return result
```



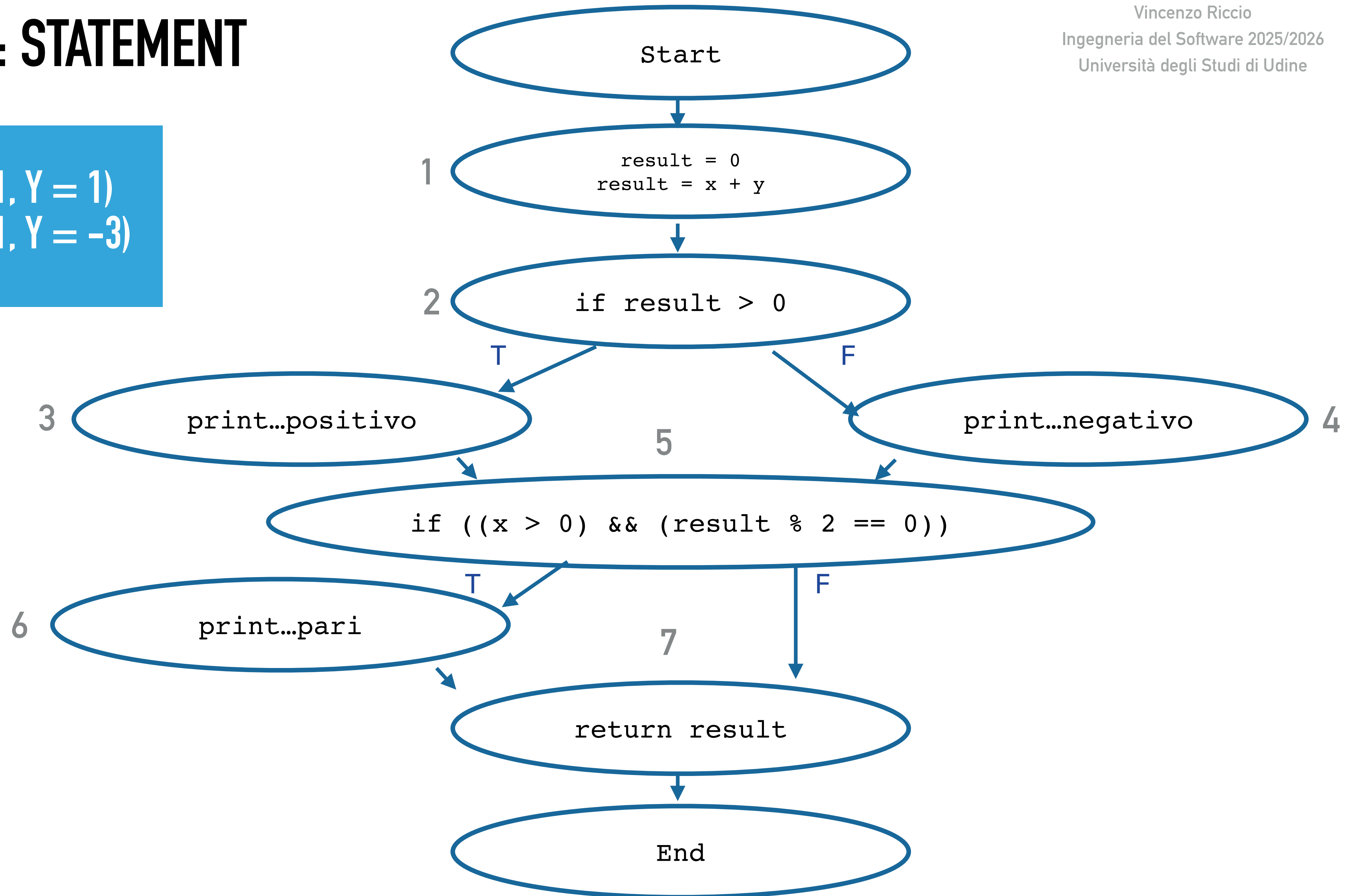
```
def esempio_control_flow(x, y):  
    # N1 - Blocco di comandi 1: Assegnamento  
    result = 0  
    result = x + y  
    # N2 - Decisione 1: Verifica se il risultato è positivo o negativo  
    if result > 0:  
        # N3 - Comando 2: Stampa il risultato se positivo  
        print("Il risultato è positivo:", result)  
    else:  
        # N4 - Comando 3: Stampa un messaggio se il risultato è non positivo  
        print("Il risultato non è positivo:", result)  
    # N5 - Decisione 2: Verifica se il risultato è pari o dispari  
    if ((x > 0) && (result % 2 == 0)):  
        # N6 - Comando 4: Stampa un messaggio se il risultato è pari  
        print("Il risultato è un numero pari e il primo addendo positivo.")  
    # N7 - Comando 6: Restituisce il risultato  
    return result
```

ESEMPIO: CFG



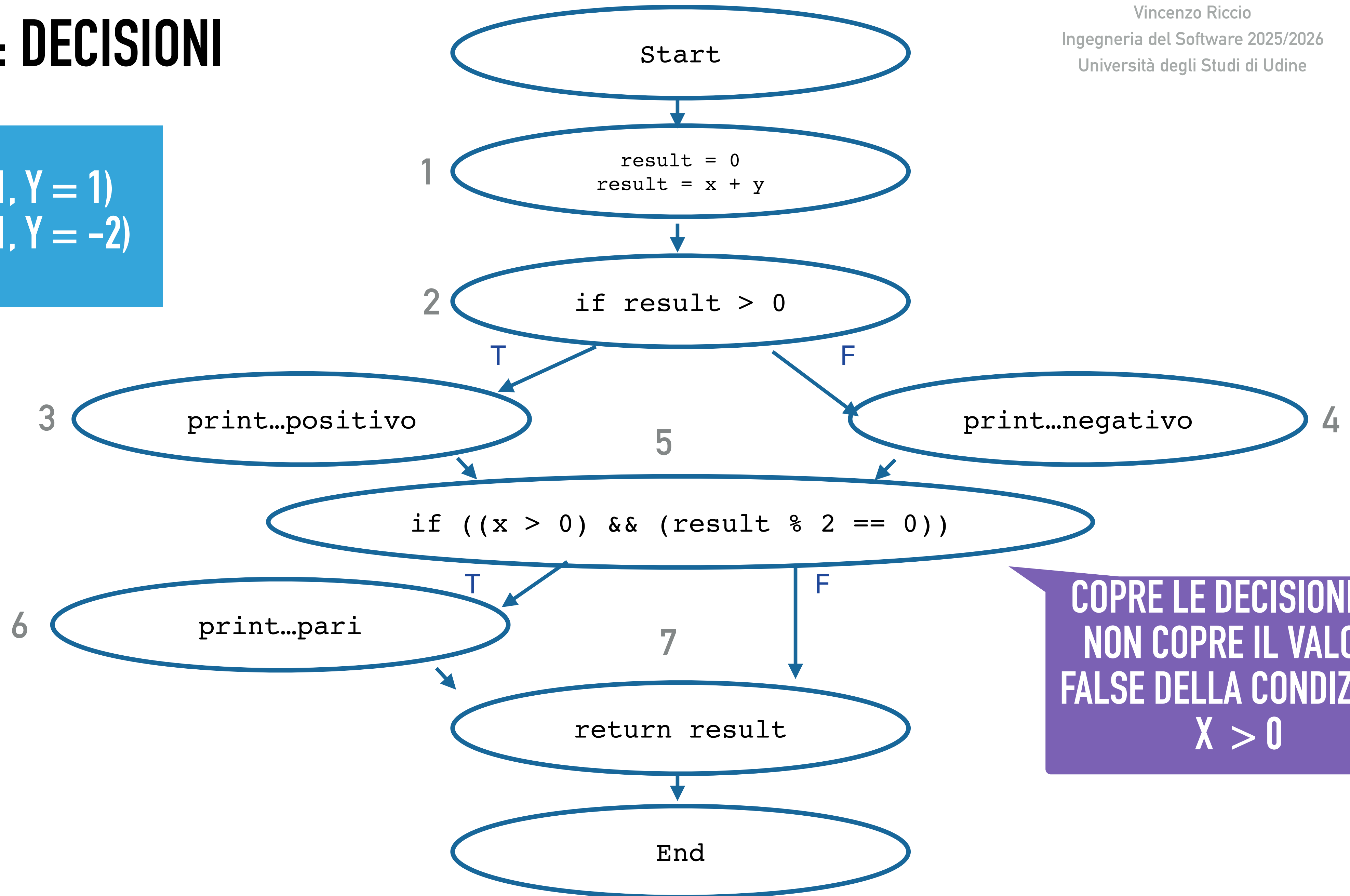
ESEMPIO: STATEMENT

TC1 = (X = 1, Y = 1)
TC2 = (X = 1, Y = -3)



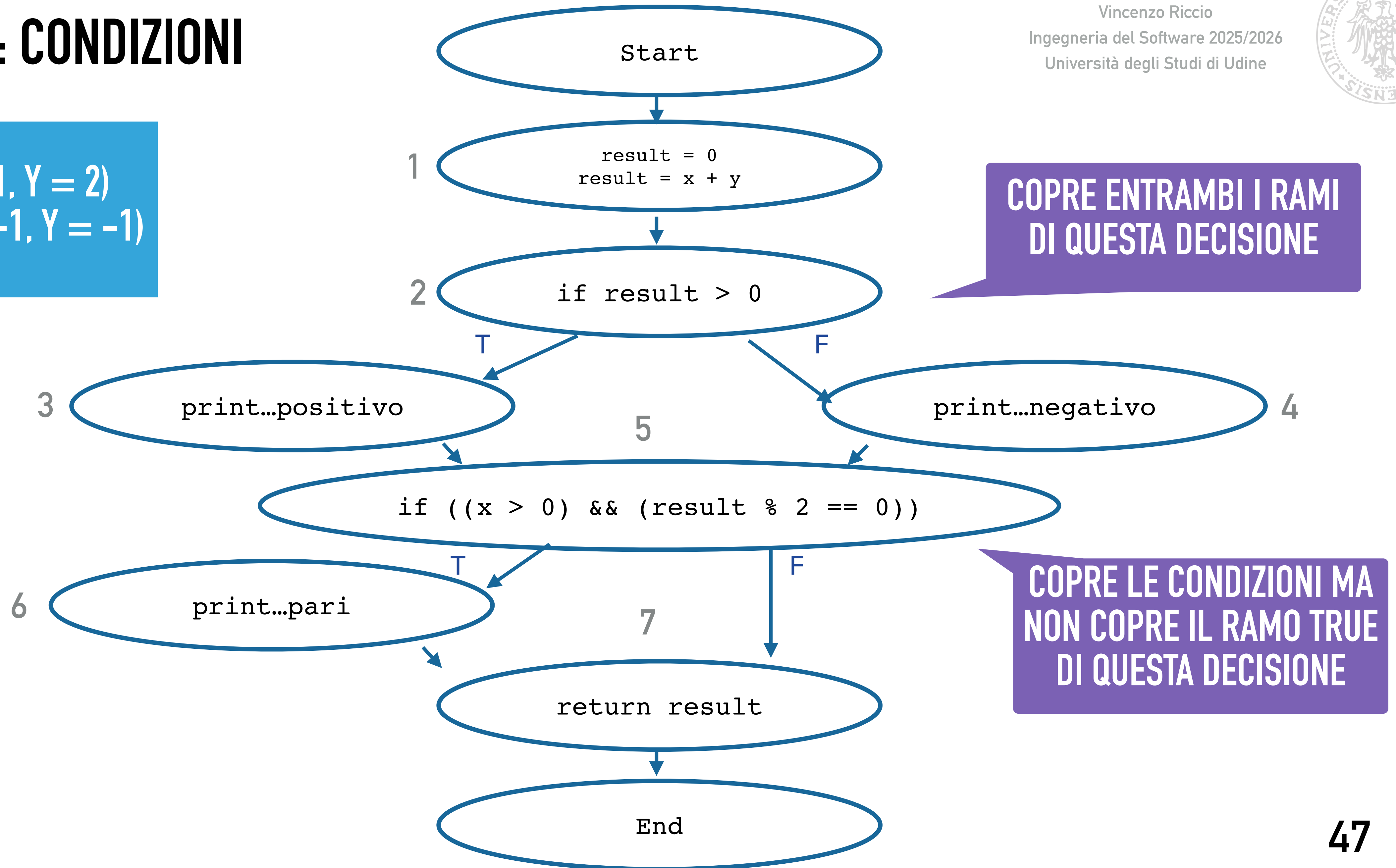
ESEMPIO: DECISIONI

TC1 = (X = 1, Y = 1)
TC2 = (X = 1, Y = -2)



ESEMPIO: CONDIZIONI

TC1 = (X = 1, Y = 2)
TC2 = (X = -1, Y = -1)



ESEMPIO: CONDIZIONI E DECISIONI

TC1 = (X = 1, Y = 1)
TC2 = (X = -1, Y = -2)

